Kavach: Deterministic-Primary Detection for LLM Agent Tool Calls

A Rearchitecture, a Real-Trajectory Validation, and Negative Results

Anonymous Author(s)

Abstract

LLM agents increasingly act in the world through tool calls, but deployed guardrails overwhelmingly evaluate an agent's language rather than the resolved arguments of the action it is about to take. We present Kavach, a runtime monitor that intercepts every tool call at the moment its arguments are resolved, via a parliament of four domain-specialized ministers — VAULT (credentials), EXECUTOR (dangerous code), CHANNEL (exfiltration), and NAVIGATOR (unauthorized action, the least mature of the four and not yet independently validated) — whose verdicts a deterministic speaker combines into Block, Escalate, or Allow. Four contributions. First: four independent lines of evidence show deterministic parsing over resolved arguments outperforms cosine similarity over semantic embedding when the attack signal is structural, moving three of four detectors to deterministic-primary detection. Second, real deployment numbers: 544/544 on InjecAgent's data-stealing corpus via CHANNEL's taint mechanism (direct-harm open: 0% per minister, 3.1% via TRAJECTORY); 5/5 AgentDojo attacks escalated with zero false blocks on 22 benign sessions; and, on a 519-case suite for VAULT/EXECUTOR (InjecAgent/AgentDojo barely exercise either), 74.4%/90.1% detection with zero new false positives, over half escalate-only. Third: replaying all 629 real recorded GPT-4o attack trajectories AgentDojo publishes, Kavach fires on 74.2%, tracing every apparent miss to its actual tool-call content. Fourth, negative results in full: failed authorization heuristics, an abandoned deterministic swap, independent generalization checks against both an author-written novel vocabulary and an externally published attack corpus, and a benign-side gap in CHANNEL's taint mechanism found and fixed by direct measurement. No GPU required; fully reproducible offline.

1 Introduction

LLM agents increasingly act in the world through tool calls: reading files, sending money, messaging other people, executing code. The guardrails deployed to secure this behavior overwhelmingly evaluate the agent's *language* — its input prompt, its chain-of-thought, its output text — rather than the *resolved arguments* of the action it is about to take. This is a real gap: a tool call to `send_money` with an attacker-substituted recipient IBAN, or to a credential-harvesting shell one-liner, is often indistinguishable from a benign call at the language level and only becomes distinguishable once its concrete arguments are inspected. Kavach is a runtime monitor that closes this gap by intercepting every tool call at the moment its arguments are resolved and scoring the resulting action — not the prompt, not the plan, the resolved call itself — before any side effect occurs.

This paper is substantially a report on a design decision and the evidence that produced it: we built Kavach's first version as a semantic, embedding-based detector — a parliament of domain-specialized ministers, each retrieving against a curated corpus of attack-pattern descriptions – and over the course of developing and stress-testing it, accumulated four independent lines of evidence that this design was the wrong primary mechanism for three of its four detection domains, while remaining the right mechanism for the fourth. We report both the evidence and the resulting rearchitecture, including two attempts that did not work, because a paper that reports only the final, working configuration understates how the field should update on evidence like this: not every detection problem shaped by an attacker's resolved arguments is best solved by comparing embeddings.

Contributions. **First, a methodology finding: deterministic parsing beats cosine similarity when the attack signal is structural (§3).** A lexical-inflation bug, a dense fine-tune that traded one failure mode for a worse one, a corpus-coverage gap that generalized past its original motivating case, and a proposed fifth minister that we formally falsified before building — paired legitimate/malicious tool calls that differ only in argument content show near-zero cosine similarity deltas (0.003–0.024) — together make the case that credential-path shapes, network-fetch-to-disk patterns, and destination-value provenance are answered more reliably by deterministic parsing over an action's resolved arguments than by any refinement of embedding similarity over its semantic content. We moved three of Kavach's four ministers (VAULT, EXECUTOR, CHANNEL) to deterministic-primary detection on this evidence, keeping cosine similarity only as an escalate-only triage layer over the deterministic residual.

Second, a deployment with real detection numbers (§3, §4). InjecAgent and AgentDojo, the standard benchmarks we also evaluate against below, almost entirely miss VAULT and EXECUTOR — both stress CHANNEL specifically, a gap found live while validating this rearchitecture. We assembled a composite evaluation corpus (§4.2) for VAULT and EXECUTOR from real, externally-validated attack data (AtomicRedTeam, GTFOBins) rather than authoring new synthetic attacks risking unconscious shaping toward the evaluating system; CHANNEL and NAVIGATOR are evaluated separately on InjecAgent and AgentDojo below. Full numbers and baselines are in §4: VAULT 74.4%, EXECUTOR 90.1%, zero new benign false positives, 544/544

on InjecAgent's data-stealing corpus via CHANNEL's taint mechanism (direct-harm remains open), and 5/5 structurally applicable AgentDojo attacks correctly escalated with zero false blocks on 22 real benign sessions.

Third, a large-scale real-trajectory validation (§4.6). Beyond benchmark replay, we validated Kavach against all 629 real recorded GPT-4o attack trajectories published by AgentDojo — actual model-generated tool-call sequences with real tool outputs, not a benchmark's minimal answer key. Kavach fires on 467/629 (74.2%); tracing every apparent miss to its actual tool-call content, most reflect the model never taking any consequential action at all (an artifact of AgentDojo's own success-labeling convention, not a detection gap), leaving 19 genuine misses, each categorized precisely by mechanism (§4.6).

Fourth, honestly reported negative results (§6). NAVIGATOR, the one minister not moved to deterministic-primary detection, is reported in full: an abandoned deterministic swap, and three independently designed session-level authorization heuristics that each failed a frozen discriminability criterion, for three distinct reasons. Independent generalization checks against both a fictional agent vocabulary and an externally-published attack corpus establish precisely which mechanisms transfer and which do not — EXECUTOR cleanly, VAULT mostly, CHANNEL's escalation path but not its hard-block path, NAVIGATOR not at all (§6). Knowing precisely where each mechanism does and does not generalize is what distinguishes known limits from limits not yet found.

Scope and non-claims. We do not claim Kavach prevents every class of agent-side attack. Its deterministic rules are authored against published taxonomies and known attack corpora; a genuinely novel technique outside both may not be caught, characterized precisely by the generalization audit (§6) rather than left as an assumption. NAVIGATOR's authorization question — was this action derivable from what the user actually asked for — remains open past a validated taxonomy classifier; we report the attempts and failure modes rather than a completed mechanism. We do not propose Kavach as a replacement for input-time or output-time guardrails; it enforces at a different lifecycle boundary, and a comprehensive deployment runs more than one.

Reproducibility. The parliament service, the deterministic prefilters and their test suites, the reference-corpus evaluation harness, and the real-trajectory replay scripts used for every number in §4 run entirely offline on CPU-only hardware, with no GPU or paid API access required.

2 Background

2.1 The agent infrastructure attack surface

Modern LLM agents are composite systems: a language model provides the reasoning surface, while the harness provides tool dispatch, context management, multi-turn state, and persistence. In production deployments the harness accounts for the overwhelming majority of source lines, with the model contributing only a small fraction — a structural inversion with a direct security consequence: the harness, not the model, is the dominant attack surface.

Agent harnesses expose several classes of latent vulnerabilities. *Configuration-as-code* attacks exploit declarative configuration the harness loads and executes without sanitization (CVE-2026-21852 on Claude Code's `.claude/settings.json`; CVE-2026-21858 on n8n). *Serialization-injection* attacks exploit the harness's serialization/deserialization of tool responses and agent state, enabling arbitrary class instantiation in trusted namespaces (CVE-2025-68664 on LangChain Core, CVSS 9.3). *Hook-bypass* attacks exploit incorrectly wired security hook points (OpenClaw issues #5513, #5943, where `before_tool_call` is documented but never invoked), and *CSRF/session* attacks exploit the harness's web surface (CVE-2025-34291 on LangFlow, actively exploited in the wild as of January 2026 [7]).

These vulnerability classes correspond directly to Kavach's minister structure: EXECUTOR detects code-execution-class attacks (CVE-2025-59536, CVE-2025-34291, CVE-2025-68664), VAULT credential-theft-class attacks (CVE-2026-21852, CVE-2025-68664), CHANNEL exfiltration-class attacks (CVE-2026-21852 via URL redirection, CVE-2026-21858), and NAVIGATOR trajectory-manipulation attacks spanning multiple tool calls. Section 3 gives the MITRE ATT&CK and OWASP Agentic 2026 taxonomic grounding for each minister.

2.2 Existing guardrails and their enforcement points

The guardrail literature can be organized along two axes: (1) *where* in the agent lifecycle the guardrail enforces, and (2) *what mechanism* it uses to detect violations.

Input-time guardrails. (PromptGuard 2 [5], Azure Prompt Shields) operate on the user's message before the agent processes it, and cannot detect attacks arriving via tool outputs, the dominant vector in prompt-injection benchmarks [21].

Plan-time / chain-of-thought guardrails. (AlignmentCheck [5], DRIFT [15]) audit the agent's reasoning trace before it commits to an action — powerful but expensive (full LLM inference), and blind to a tool call's *resolved arguments*, seeing only declarative intent.

Tool-call-time guardrails. operate at the tool-call boundary — Kavach's enforcement point. CodeShield [5] applies Semgrep to code-returning outputs only; ShieldAgent [4] retrieves rule circuits per trajectory; Task Shield [12] checks task alignment. None uses a curated semantic corpus of attack-pattern descriptions at the tool-call level.

Information-flow guardrails. (CaMeL [8], FIDES [6], RTBAS [23]) tag data with provenance/capability labels and enforce statically — provable, but unable to detect attacks whose provenance is correctly labeled while the argument content itself is malicious.

Output-time guardrails. (NeMo Guardrails, Conseca [19], and the post-hoc monitor Kavach replaces) act after execution — useful for forensics, unable to prevent it.

Drift-detection guardrails. (TaskTracker [1], DeepContext [3], Trajectory Guard [2]) measure trajectory divergence from stated intent across turns. Kavach's NAVIGATOR minister sits here at the per-call level, comparing a proposed consequential action against the session's pinned user intent; we report both a completed taxonomy layer and an incomplete authorization mechanism for it in §6.

Across these categories, the tool-call boundary is the only enforcement point at which the *resolved runtime arguments* of an action are available for inspection before that action executes. Input-time and plan-time guards see only declared intent — the user message or the agent's reasoning trace — not the concrete arguments the call will run with; output-time guards observe the action only after it has executed, too late to prevent it; and information-flow guards enforce data provenance rather than the semantics of the argument content. This is Kavach's enforcement point, and it is structurally unreachable by guards in the other categories: they fire at boundaries where the resolved-argument view does not yet, or no longer, exists. §6 reports a consistent, if narrower, coverage gap for the two closest available tool-call-content tools specifically, not a test of the general structural claim above.

Table 1 (§5) places Kavach in context on five axes: enforcement boundary, mechanism, corpus extensibility, verdict combination, and reported AgentDojo / InjecAgent performance.

2.3 Benchmarks and evaluation methodology

The three most widely used agent-security benchmarks are:

InjecAgent [21] — 1,054 prompt-injection cases (17 user tools × 62 attacker tools) split into direct-harm (30 tools) and data-stealing (32 tools) settings, each pairing a benign user instruction with a malicious tool-output payload and an expected attacker action. Kavach runs the 544-case data-stealing (DS) subset cold, and the direct-harm subset separately (§4.7).

AgentDojo [9] — 97 user tasks and 629 security test cases across four environments, scored by ASR and utility. We evaluate against its structurally applicable ground-truth attack cases (§4.5) and, at scale, against all 629 real recorded GPT-4o attack trajectories it publishes (§4.6).

A methodological note: reporting attack recall without the accompanying benign false-positive rate conflates detector quality with aggressiveness. We therefore report both jointly for every detection number in §4, following Kapoor et al. [14], rather than gating recall behind a fixed false-positive ceiling.

2.4 OWASP and MITRE taxonomic grounding

Kavach's attack-pattern corpus is authored against the two most widely cited AI-agent threat taxonomies, not against attacks the team happened to have seen: MITRE ATT&CK/ATLAS (EXECUTOR: T1059, T1546; VAULT: T1552, T1539; CHANNEL: T1041, T1567; NAVIGATOR: T1083 and others) and the OWASP Top 10 for Agentic Applications 2026, mapped per minister (EXECUTOR: ASI04/05; VAULT: ASI03/09; CHANNEL: ASI07/08; NAVIGATOR: ASI01/10).

3 Kavach Architecture

3.1 Overview

Kavach is a runtime guardrail that interposes between an LLM agent's tool dispatch and the tool's actual execution. Every resolved tool call is evaluated by a parliament of four domain-specialized ministers — VAULT (credential and identity abuse), EXECUTOR (dangerous code execution), CHANNEL (data exfiltration and provenance), and NAVIGATOR (unauthorized or drifted agent action), pre-selected per call by a similarity-gated router (falling back to all four when none crosses its threshold). VAULT, EXECUTOR, and CHANNEL are validated detectors with reported detection numbers throughout this paper; NAVIGATOR is the fourth, exploratory component, and we report plainly that it is not yet one (§6) — every aggregate figure including a NAVIGATOR contribution reports it broken out per-minister, not folded in invisibly (e.g. §4.7, §4.8). A deterministic Speaker combines their verdicts by pure veto (any minister's BLOCK suffices, no score averaging) into BLOCK, ESCALATE, or ALLOW, via a single HTTP endpoint, `POST /hook/parliament`, called synchronously at the host runtime's pre-execution hook; a verdict other than ALLOW prevents the call from executing. Two further pre-rearchitecture components remain outside the four ministers: TRAJECTORY, a session-level monitor re-confirming or escalating a subset of already-detected cases (disclosed in full, including a real benign false-positive interaction, in §3.8); and COMPASS, a session-intent-drift check that *can* independently upgrade a verdict to BLOCK when active, but whose trigger condition never arises in any of this paper's dispatch harnesses — confirmed zero occurrences across all 519 + 1,054 + 629 evaluated cases (§4.8).

3.2 Deterministic-primary detection: why, and for which ministers

Kavach's ministers were originally cosine-primary: each proposed action was embedded and scored against a curated corpus of attack-pattern descriptions, with a per-minister confidence threshold deciding the verdict. Four independent lines of evidence, developed over the course of this project, converged on replacing this design for three of the four ministers.

- **Evidence 1 — a lexical-inflation bug pointed at the fix, not just the bug.** BM25/IDF term weighting over-weighted rare generic tokens (`url`, `user`)

because L3 code-syntax text had leaked into the lexical index, false-blocking benign calls (`get_webpage`, `read_inbox`). A regex/exact-match layer on L3 syntax tokens shipped instead of two rejected rerankers (SPLADE, cross-encoder): the false positives were *structural*, and a deterministic match on structure resolved them without touching the semantic layer.

- **Evidence 2 — a targeted dense fine-tune traded one failure mode for a worse one.** A low-rank adapter fine-tuned on the frozen BGE encoder fixed the false positives outright (71%→0%) but regressed real attack recall on 30 genuine InjecAgent attacks from 63.3% to 26.7%: its LOLBIN-shaped training positives sharply separated *that* shape and pulled genuinely different attack shapes (bank transfers, access-grant abuse) away from the decision boundary. The adapter was correctly not merged.
- **Evidence 3 — the corpus gap this exposed was general, not a training artifact.** Adding 62 real business-logic attacks (transfers, access grants) as positive examples revealed they do not match *anything* already in the corpus (mean cosine similarity 0.565, far below any operating threshold) — a structural blind spot for an ordinary-looking business action carrying a malicious argument.
- **Evidence 4 — a proposed fifth minister was formally falsified before being built.** We tested whether cosine similarity could discriminate STEWARD's target attack class at all, using paired legitimate/malicious AgentDojo calls sharing a tool name and differing only in argument content: similarity deltas were near zero (0.003–0.024). STEWARD was not built — argument-*value* provenance is a different question than action-level semantic similarity, and no corpus expansion closes that gap for a mechanism that only looks at the action as a whole.

The decision. Evidence 1–4 point at one distinction: structural attack signals (a credential-path shape, a network-fetch-to-disk pattern, an argument value's provenance) are caught more reliably by deterministic parsing than by any refinement of cosine similarity, while genuinely intent-shaped signals may still need a semantic or session-state mechanism. We therefore moved VAULT, EXECUTOR, and CHANNEL to deterministic-primary detection, keeping cosine only as an escalate-only triage layer on the deterministic residual (§3.5). NAVIGATOR remains cosine-primary; a direct deterministic swap was attempted and abandoned at its own go/no-go gate (zero net discrimination gain), and a second attempt at a deterministic authorization layer is reported as a negative result in §6.

3.3 VAULT and EXECUTOR: structural parsing

VAULT and EXECUTOR score each proposed tool call's arguments with deterministic prefilters (`parliament/prefilters.py`) built on a behavioral-primitive taxonomy: VAULT's primitives (V1–V6) cover credential paths, dotenv files, and secret environment variables; EXECUTOR's (E1–E7) cover network-fetch-to-disk, reverse-shell shapes, and persistence writes. Two extraction layers back these, chosen per shell dialect by a latency/correctness trade-off measured directly rather than assumed:

Real AST parsing where it is both correct and fast. For `sh`/`bash`, arguments are parsed with `bashlex` into a real syntax tree, closing shape-based false negatives a tokenizer cannot (e.g. heredoc content) at ∼0.5 ms per call. PowerShell has an equivalently real AST, but invoking it needs a `powershell.exe` subprocess at ∼1.4 s per call — 18× over budget — so the real AST is wired in only for `sh`/`bash`; PowerShell and `command_prompt` remain on the tokenizer below, with the real PowerShell AST kept as an offline/audit tool.

A structural tokenizer for the remaining dialects. For dialects where a real grammar parser is unavailable or too slow, a lightweight shell-quote-aware tokenizer extracts URL-, path-, and credential-shaped tokens directly, with a two-tier credential test (a strong extension/path match, and a broader path-shaped fallback) rather than a single regex.

Full detection numbers, the deterministic/cosine Block/Escalate split, benign false-positive rates, and a held-out-split stability check are reported in §4.3, together with the per-minister benign escalate rates on both populations.

3.4 CHANNEL: session-level taint and value provenance

CHANNEL is inherently a multi-call, session-level detector: the attack shape it targets is a sensitive-data *read* followed by an external *send* in the same session, which no single-call regex can express. It maintains two independent pieces of per-session state.

Taint. A call to a known source tool (a sensitive-data read) marks the session tainted; a subsequent call to a known destination tool is BLOCKed if tainted and the destination is not allow-listed — the allow-list is empty by default, since no corpus evidence supports a "trusted" external recipient distinct from an attacker-controlled one. Source and destination are matched by tool name against curated lists, extended with a tokenizer and a behavior-general fallback checking destination-shaped values (email, IBAN, URL) for any unnamed tool, so a genuinely novel exfiltration tool is not silently invisible.

Provenance. Independent of taint, every destination-tool call's argument values are classified by where they came from this session: USER_SUPPLIED, SELF (verified identity), OUTPUT_DERIVED (first seen in an earlier tool call's output this session), or NOVEL (neither). OUTPUT_DERIVED or NOVEL on a consequential call returns ESCALATE, deliberately not BLOCK: paying a bill by reading its own file for the recipient IBAN, or replying to an email just read, are real, legitimate same-session read-then-send chains that must not be

treated the same as an attacker-substituted destination (§3.8). Taint's BLOCK authority previously ignored this classification entirely, hard-blocking legitimate self- or user-named destinations we found and fixed by direct measurement (§4.7); a destination first seen only in a prior tool's *output* still blocks, deliberately, since that is exactly how an injected destination would appear too.

CHANNEL's locked reference numbers and full evaluation methodology are reported in §4.5.

3.5 Cosine-ESCALATE-only triage

For VAULT and EXECUTOR, cosine similarity against the original attack-pattern corpus is retained, but only as a triage layer over the deterministic *residual* — cases the structural parsers did not match — and its verdict is hardcoded to ESCALATE: it can never BLOCK. A semantic signal that cannot reliably discriminate (Evidence 4) should not be trusted with an irreversible verdict, but remains useful as a flag for human review on cases a deterministic rule did not anticipate.

3.6 An LLM tiebreaker for narrative-phrasing attacks (built, not deployed)

VAULT and EXECUTOR's deterministic rules match literal syntax; an attack described in prose rather than code ("append an entry to the SSH authorized-keys file for persistence" rather than the literal path) can evade every rule that only recognizes concrete command shapes. We built an escalation tier for this residual that keeps the verdict-making step model-free even though it consults a model: a cheap deterministic pre-flag decides whether a call is worth escalating at all, a small local model (`qwen2.5:7b`) only *extracts three narrow factual flags* from flagged calls rather than rendering a verdict directly, and a separate deterministic rule converts those flags into BLOCK/ESCALATE/ALLOW (failing open to ALLOW on model timeout or error). Two simpler designs — raw entity-proximity matching and asking the model to judge malicious-or-benign directly — were tried and rejected first, for benign false positives and missed attacks respectively. Validation results, including a generalization check and why it is held back from live dispatch, are in §4.4.

3.7 Provenance ledger

Every verdict is logged to an append-only, SHA-256 hash-chained SQLite ledger that is tamper-*evident*, not tamper-*proof*; full mechanism and a real verification-bug episode found and fixed during this work are described in Appendix C.

3.8 Design decisions and trade-offs

Deterministic-primary is a per-minister decision, not a blanket policy. NAVIGATOR's question — was this action authorized by what the user actually asked for — does not reduce to a structural shape the way a credential path or network-fetch-to-disk pattern does, so it was not moved uniformly with the other three (§6). CHANNEL's provenance mechanism escalates rather than blocks for the same kind of reason: real, common, benign same-session read-then-send patterns in actual trajectory data would be incorrectly stopped by a hard block, so we chose the reversible verdict where that false-positive cost is concrete.

Escalate has a real operational cost, and we report it as such. An Escalate verdict is a request for human review, not a prevented action — the call still reaches the user or operator queue, and someone has to look at it. This has a real cost on legitimate traffic: on the 55-case agent-shaped benign population, 37/55 (67.3%) trigger at least one Escalate somewhere in the pipeline — a majority of ordinary calls, not a tail case. VAULT and EXECUTOR's full-pipeline detection numbers (§3.3) report the Block/Escalate split explicitly for exactly this reason: a reader who reads either recall number as "prevented" without checking that split will overstate what the deployed system prevents versus flags for review. A high benign Escalate rate also raises whether an Escalate is informative at all; §4.3 reports Escalate-precision directly rather than leaving that question open.

Synchronous evaluation on the hot path. Kavach evaluates synchronously in the pre-execution hook. Asynchronous evaluation would reduce latency but allow a race between the verdict and execution; we accept the latency cost to maintain the guarantee that no tool executes before a verdict is issued.

A pre-rearchitecture session monitor remains active alongside the four ministers. TRAJECTORY predates this rearchitecture (§3.2) and combines five signals — per-call confidence, session-intent drift, action coherence, denial-echo, cross-minister escalation — into a per-session risk score, sharing its mechanism with the intent-drift detector reported as a negative result in §6. It is not a first-line detector: zero cases counted toward a reported detection number were first identified by TRAJECTORY across every locked evaluation — a minister always fires first. It instead re-confirms or escalates already-detected cases: 148 of the 629 real GPT-4o trajectories in §4.6 (none affecting the reported 467/629), and 3 of the 22 real benign AgentDojo sessions in §4.5 — a hard BLOCK, jointly with NAVIGATOR, on benign traffic our escalate-only CHANNEL mechanism does not produce. We report this plainly: a real false-positive-producing interaction on a live component not subject to the same validation rigor as VAULT, EXECUTOR, and CHANNEL, and not a contribution of this paper.

4 Evaluation

We evaluate Kavach on three independent axes: deterministic-primary detection on a composite evaluation corpus for VAULT and EXECUTOR, assembled because the two standard benchmarks below almost entirely miss them (§4.2–4.3); CHANNEL's session-level provenance mechanism against real multi-step AgentDojo trajectories (§4.5); and a large-scale validation against AgentDojo's own recorded model transcripts (§4.6). We report benign false positives jointly

with recall throughout, and distinguish in-process from live-HTTP-dispatch measurements, since these diverged in ways that mattered (§4.1).

Consistent with the ACM Policy on Authorship, generative AI tooling (Anthropic's Claude, via Claude Code) was used to implement detection mechanisms from the authors' designs, build and run the evaluation harnesses, and assist analysis; all methodology, architecture, and interpretation were specified and verified by the authors. Full disclosure follows the references.

4.1 An audit discrepancy, and how it was resolved

Early in this work, three measurements of VAULT/EXECUTOR detection rate on the same reference corpus disagreed sharply: 34.2%/34.5% (deterministic layer alone, in-process, since superseded by a VAULT rule fix to the current 35.9%/34.5%, §3.3), 72.6%/89.9% (full pipeline, in-process), and 14.9%/19.2% (an earlier live-HTTP measurement). Two independent causes were responsible: the live-HTTP figure was corrupted by a duplicate server process answering a fraction of requests, and the metric itself differed — one measurement counted Block-or-Escalate as a hit, another counted only the deterministic layer's contribution. Once corrected, the live-HTTP and in-process full-pipeline numbers agreed — a reminder that a benchmark harness's own bugs can masquerade as detector performance.

4.2 A composite evaluation corpus for VAULT and EXECUTOR

InjecAgent and AgentDojo, the two standard agent-security benchmarks we rely on elsewhere in this section, almost exclusively stress CHANNEL: InjecAgent's attack shape is a read-then-send taint chain, and AgentDojo's is a provenance-mismatched destination value on a consequential call. Neither meaningfully exercises VAULT or EXECUTOR at all — a gap we found live, while validating VAULT/EXECUTOR's deterministic-primary rearchitecture (§3.2) and discovering we had no real benchmark to validate it against.

Rather than author new synthetic attacks to close this gap, we assembled a unified, composite evaluation corpus for VAULT and EXECUTOR from already-real, externally-validated attack data (AtomicRedTeam, GTFOBins, BFCL) under one runner, one wire format, and one scoring convention. This is deliberate: a self-authored attack set risks unconsciously being shaped to be catchable by the very system that writes it, and reusing independently-authored corpora avoids this risk structurally rather than by discipline alone.

The corpus's other 207 cases, and Kavach's rate on all 519. The remaining 207 cases (all AtomicRedTeam) target CHANNEL (72) or NAVIGATOR (144), not VAULT/EXECUTOR; their *primary* evaluation remains InjecAgent (§4.7) and AgentDojo (§4.5, §4.6), but we report this in-corpus subset rather than leaving it unaccounted for. Any-minister detection on these 207 is 206/207 (99.5%), though each minister's own verdict alone catches only 23/72 (31.9%, CHANNEL) and 92/144 (63.9%, NAVIGATOR) — most of the remainder is caught by cosine-triage or a different minister, not the targeted one's primary mechanism. Kavach's overall any-minister, any-target rate across all 519 — the same denominator the baselines below use — is 515/519 (99.2%); this is a looser metric than the strict per-target recall reported next, and should be read as such, not as a like-for-like comparison.

Reuse, unification, and scoring. Attack cases are drawn from AtomicRedTeam and GTFOBins; benign calls from BFCL (793, general-purpose) plus a 55-case agent-shaped benign set, reported separately rather than pooled. What is novel is the unification: a single harness dispatching every case through the same `/hook/parliament` path, one per-minister scoring convention, and one benign false-positive convention. Each attack case carries a `target_patterns` list naming which minister(s) it should trigger; we report both an overall detected rate and a per-minister hit rate, since a case can be detected overall while its targeted minister misses. On the benign side, Block is a hard false positive and Escalate is reported separately, never conflated with Block or a clean Allow — the same convention CHANNEL's provenance mechanism uses (§3.4).

4.3 Results: VAULT and EXECUTOR

On this 519-case AtomicRedTeam/GTFOBins reference corpus (117 VAULT-targeted, 203 EXECUTOR-targeted; `target_patterns` is unmodified since its original authoring commit, confirmed via `git diff`, so it is independent ground truth), the deterministic layer alone recovers VAULT 35.9% (42/117) and EXECUTOR 34.5% (70/203); adding the cosine-escalate-only triage layer (§3.5) raises full-pipeline detection to VAULT 74.4% (87/117) and EXECUTOR 90.1% (183/203) — of which 42 points (VAULT) and 70 points (EXECUTOR) are hard-Block from the deterministic layer, and 45 and 113 points respectively are Escalate-only from cosine triage — with zero new hard-block false positives on either benign population (793 BFCL, 55 agent-shaped). Benign Escalate rates on the two populations (agent-shaped/BFCL) are: VAULT 5.5%/13.4%, EXECUTOR 0%/14.9%, CHANNEL 40.0%/4.5%, NAVIGATOR 21.8%/10.8%. A held-out even/odd split confirms the full-pipeline rate is stable, not an artifact of partition: VAULT 71.2% (42/59) versus 77.6% (45/58); EXECUTOR 89.2% (91/102) versus 91.1% (92/101) — both halves reported explicitly rather than only their average.

Escalate-precision, not just recall. A high benign Escalate rate could make an attack-side Escalate uninformative, so we report Escalate-precision — of a minister's own Escalate verdicts, what fraction land on a true attack — per benign population rather than pooled: VAULT is 93.8% (45 true vs. 3 false) on agent-shaped benign but 29.8% (45 vs. 106) on the larger, more general BFCL population; EXECUTOR is 100% (113 vs. 0) agent-shaped but 48.9% (113 vs. 118) on

BFCL. Precision is high on the population closer to Kavach's deployment context and lower on generic traffic, an effect of BFCL's volume amplifying a modest per-call rate rather than a change in the mechanism; both populations' benign:attack ratios are corpus-constructed, not a measured production base rate, so these figures should be read relative to each other, not as an absolute expectation.

For comparison, the pre-rearchitecture cosine-primary configuration — same harness, same 519 cases, same scoring convention, code unmodified from before the rearchitecture — recovers VAULT 67.5% (79/117) and EXECUTOR 85.7% (174/203) target-minister recall, but with a 4.9% (39/793) hard-block false-positive rate on general-purpose benign (BFCL) traffic. The deterministic-primary pipeline exceeds cosine-primary's recall on both ministers (74.4%/90.1% vs. 67.5%/85.7%) while eliminating that false-positive rate entirely (0% hard-block FPR on both benign populations) — both an ablation for the paper's central deterministic-vs-cosine claim and a same-team, same-harness baseline showing the rearchitecture strictly dominates the design it replaced.

We additionally compare against two independent, non-Kavach baselines on the identical 519-case corpus and both benign populations, matching their denominator against Kavach's own 515/519 (99.2%, §4.2) rather than the narrower per-target figures above. A naive keyword/regex blocklist (∼30 hand-written patterns, no corpus, no tuning) recovers 12.1% (63/519) at 0% FPR — the expected floor for an uncorpus-driven list. A zero-shot LLM judge (local `qwen2.5:7b`, no corpus, single fixed prompt, resolved call only) recovers 78.8% (409/519), well below Kavach's 99.2% on the same 519 cases, at a real false-positive cost Kavach's deterministic layer avoids entirely: 1.4% on BFCL and 14.5% on agent-shaped calls specifically. A larger model (`qwen2.5:32b`, same prompt and populations) does not simply dominate: recall falls to 71.3% (370/519) while FPR drops to 0.4% combined (0% agent-shaped) — a different point on the same recall/precision trade-off, not a strictly better one, indicating the trade-off is inherent to LLM-judge-style detection rather than an artifact of model scale. Read together with the cosine-primary ablation above: a naive filter under-detects, a generic LLM judge (at either scale) trades recall against precision but cannot match both, and cosine-primary approaches neither Kavach's recall nor its zero-FPR guarantee.

This split addresses partition luck, not rule-authoring circularity: the deterministic rules were authored by inspecting misses across the full corpus, so both halves were visible during rule development. The generalization audit (§6.1) attempts to measure past this limit, and has its own limit in turn: its scenarios are author-written, testing vocabulary novelty rather than an independently sourced distribution.

4.4 LLM tiebreaker: validated, deliberately not deployed

The tiebreaker (§3.6) scored 16/17 (94.1%) in pilot. Revalidated against all 144 real benign-set calls, it produced one new false positive (a `non_standard_source` misread of legitimate, locally-stored content); a prompt clarification fixed this with zero new false positives on re-test. A generalization check — five phrasings never used to write the fix, plus three negative controls — then scored 4/5, with one confirmed remaining miss (an internal file run "from the ops share," a network-share phrasing the fix does not cover). We stopped here rather than re-running against our full narrative-phrasing test set: a mechanism with one known, characterized generalization gap should not be scored as if it had none. The tiebreaker is real, working code, held back from live dispatch specifically because its own validation found a limit before we judged it decided.

4.5 CHANNEL: session-level provenance

CHANNEL's provenance mechanism (§3.4) is evaluated on two real populations: 22 real multi-step benign sessions and the AgentDojo attack cases whose two-call read-then-send shape is structurally applicable to CHANNEL's mechanism. On the benign population, 5/22 sessions correctly escalate via CHANNEL with 0 hard-block false positives from CHANNEL itself — exactly the sessions with a legitimate destination value (an IBAN, most commonly) read earlier in the same session rather than typed by the user (§3.4). A further 3/22 are hard-blocked by an unrelated TRAJECTORY/NAVIGATOR interaction, never blended into the 5/22 figure and reported in full in §3.8. On the attack side, 5/5 structurally applicable AgentDojo cases correctly escalate, verified twice: against the benchmark's minimal `ground_truth()` sequence, and independently against AgentDojo's own full recorded model trajectories for the same cases — real, longer sessions with benign calls interleaved. This second check caught a bug in our own verification script (a benign call after the attack step could overwrite an earlier correct escalation) before we reported the number as final; the corrected script confirms genuine 5/5.

4.6 Large-scale real-trajectory validation

The measurements above use either a synthetic reference corpus or a small, hand-verified set of AgentDojo cases. To validate at scale on real model behavior, we replayed all 629 real recorded attack trajectories for GPT-4o (`gpt-4o-2024-05-13`) across AgentDojo's four suites' `important_instructions` family, sourced directly from AgentDojo's own published `runs/` data — real transcripts with real tool calls and outputs, not a minimal ground-truth replay.

Headline result. Kavach fires (BLOCK or ESCALATE, any minister, at any point in the session) on 467/629 (74.2%) of these real trajectories: banking 119/144 (82.6%), slack 105/105 (100%), travel 63/140 (45.0%), workspace 180/240 (75.0%).

Diagnosing the apparent misses. Of the 162 trajectories on which Kavach did not fire, we traced every case to its actual tool-call content rather than reporting a bare miss rate. AgentDojo's own `security` field marks a trajectory as

a successful attack whenever the environment state changed at all relative to the pre-trajectory state and does not exactly match the injected goal — conflating "the injected attack succeeded" with "the model made no consequential call related to the injection at all." 131 of the 162 misses fall in the latter category, verified by extracting every tool call from each miss's transcript against a fixed consequential-tool list: the model never invoked any consequential tool in connection with the injected instruction, so there was no unauthorized action for any minister to catch, regardless of what AgentDojo's field reports. A further 12 are trajectories where AgentDojo's own field already indicates the injection did *not* succeed (8 `workspace`, 4 `travel`) — not Kavach misses either, since there is no successful attack to have caught. The remaining 19 are genuine misses.

The remaining 19 real misses, categorized by mechanism. We categorize each by whether the underlying attack has a destination *value* to trace (CHANNEL's domain) or is purely an unauthorized *action* with no data flow at all (closer to NAVIGATOR's domain, §6). Five cases (`travel`, all `reserve_hotel`) name an attacker-substituted external party — a real destination value, of the same shape as `send_money`'s recipient IBAN — but the tool was never added to CHANNEL's destination-tool list, a scope gap rather than a mechanism failure. Fourteen cases (`banking`: 11; `workspace`: 3, `delete_file` and `append_to_file`) mutate existing account or same-account state with no new destination value introduced at all — no value for any provenance mechanism to trace, since these attacks ask "should this action have happened," not "where did this value come from." Adding the tool to CHANNEL's destination list to close the five `reserve_hotel` cases introduces a new false positive instead: the value extractor recognizes only email/IBAN/phone/URL shapes, not proper-noun business names. We reverted this change and report the gap as a documented scope limitation (§6) rather than a closed fix.

Scope of this validation. This covers one model (`gpt-4o-2024-05-13`) and one attack family, the complete recorded trajectories AgentDojo publishes; transfer to other models' tool-calling styles or subtler injection framings is untested, a deliberate trade of breadth for realism.

4.7 InjecAgent

InjecAgent's full corpus is 1,054 cases: 510 direct-harm (DH) and 544 data-stealing (DS), each the cross product of InjecAgent's 17 user tasks with its 30 DH / 32 DS attacker instructions. Both are dispatched as a two-step sequence — a source read, then the destination send (DS) or the direct-harm action itself (DH) — so CHANNEL's session-scoped taint/provenance tracker can observe the read-then-act pattern in both settings; DH's dispatch was corrected to this two-step form as a methodology fix, not a change to any minister's detection logic.

On DS, Kavach achieves **544/544 (100%) strict (hard-Block) recall**, entirely via CHANNEL: every one of the 544 cases is decided by CHANNEL's taint mechanism (`CHANNEL-TAINT :read-then-send`), with VAULT and EXECUTOR returning Allow on all 544 — this result exercises CHANNEL's multi-step provenance/taint mechanism directly, not VAULT/EXECUTOR's deterministic layer.

The paired 0% benign hard-block FPR did not test this mechanism, and we found and fixed a real gap. InjecAgent's benign dispatch is single-call — no destination step follows the source-read — so taint's precondition is never presented to it; the 0% figure is a structural guarantee of the dispatch, not a measurement. Direct test: 32 genuine two-step benign actions built from InjecAgent's own tool vocabulary, dispatched live. Legitimate self-forwarding was hard-blocked 32/32 times, since taint's only carve-out was a literal account-identity match this harness never supplies. We fixed the two positively confirmable cases — destination matches the session's own identity, or was named in the user's own instruction — by having taint defer to CHANNEL's own provenance classification (§3.4); re-validated with zero change to DS's 544/544 recall or the 519-case reference corpus (checkpoint gate unchanged: 87/117 VAULT, 183/203 EXECUTOR). The harder case — a destination first seen only in an earlier tool's output, which could be a legitimate invoice recipient or exactly how an injected destination is smuggled in — remains an open gap, deliberately left blocking rather than resolved by this fix, since resolving it correctly needs the still-incomplete NAVIGATOR authorization work (§6).

DH tells a different, weaker story: no individual minister reaches hard Block on a DH case. 221/510 (43.3%) reach the loose (block-or-escalate) level, split across CHANNEL (153), VAULT (34), EXECUTOR (17), and NAVIGATOR (17); the remaining 289/510 (56.7%) are missed entirely. CHANNEL's provenance mechanism escalates 153/510 (30%) via a novel-value classification, confirming the fix surfaces the intended data-flow signal, while its own hard-block contribution stays exactly 0%, as designed. Separately, 16/510 (3.1%) reach a hard Block in aggregate via TRAJECTORY (§3.8), not any minister — consistent with its already-disclosed role, not a change to any minister's detection contract. DH remains a genuine, unresolved gap, not a strength of the current architecture, and is not folded into DS's headline number.

On the paired benign side (1,054 user-instruction prompts, one per attack case), hard-block FPR is **0%** — no benign prompt is ever hard-blocked — but the soft-escalate rate is **35.2% (371/1,054)**, again led by CHANNEL (248), with EXECUTOR (62) and NAVIGATOR (61) contributing the rest and VAULT at 0. DS's strong recall and DH's weak recall should both be read against this FPR context, not in isolation.

4.8 Cost

Kavach's latency is bimodal, and we report the split rather than a blended figure: 17 of 159 timed calls complete under 90 ms (p50 46 ms, p95 86 ms), the remaining 142 in over 680 ms (p50 733 ms, p95 1437 ms), with no call in between. The split does not cleanly track whether a deterministic

rule short-circuited the call — 26 of 159 were short-circuit-eligible and matched, of which only 17 returned fast despite an identical decision path — traced to the pipeline's ledger-completeness design: every response, short-circuited or not, spawns a background task running the skipped COMPASS (§3) and NAVIGATOR scoring for audit purposes, competing for the same CPU-bound embedding inference as the next request's foreground call under sequential dispatch. This is a scheduling artifact of our single-process CPU measurement, not the detection logic; whether bounding or deferring it in a real deployment actually closes the gap is unmeasured (§6). On GPU-equipped hardware we have separately observed the embedding path at ∼78 ms p50 (RTX 4090, steady state); verdicts are hardware-independent (replaying 107 reference cases with the encoder pinned to CPU produced zero verdict or fired-minister differences against the GPU baseline). The bashlex parse for `sh`/`bash` (§3.3) measures ∼0.5 ms per call, negligible on either path.

5 Related Work

Positioning relative to concurrent runtime work. MXC (Microsoft Execution Containers, Build 2026) and ClawGuard [22] both harden the OpenClaw runtime Kavach was originally built against. MXC is an OS-level policy sandbox — orthogonal to Kavach's argument-content question, since an authorized agent can still be manipulated into exfiltrating data within its own policy. ClawGuard's 0% AgentDojo attack-success rate is under hand-authored rules only, its rule-derivation component unevaluated. DeepContext [3] and PSG-Agent [20] are architecturally adjacent (Table 1); our differentiation is that mechanism choice (deterministic vs. cosine) is made per detection domain on direct evidence, with generalization reported explicitly (§6) rather than only success.

Plan-time authorization systems. PlanGuard [11], IntentGuard [13], and Progent [18] are the closest precedents for NAVIGATOR's authorization question, each at the plan level rather than the tool-call level. PlanGuard's hard-constraint stage plus LLM verifier reports 100% attack-blocking on InjecAgent; NAVIGATOR's analogous pinned-intent design is instead an incomplete, three-attempt negative result (§6). IntentGuard's user-confirmation-on-alert is close in spirit to CHANNEL's escalate-not-block verdict (§3.4). Progent's progressive allowlist reports an AgentDojo ASR reduction from 39.9% to 1.0%, an upper reference point, not a number Kavach reproduces.

Red-team and scanning tools. garak [10], PyRIT [16], and LLM-Guard [17] probe or scan at the prompt/input boundary pre-deployment rather than enforcing at the tool-call boundary at runtime — complementary to, not competing with, Kavach.

Table 1 (Appendix A) situates Kavach against 26 systems on five axes, drawn from each cited paper's own evaluation setup — benchmark subset, environment, and threat model all vary — so it is a map of enforcement boundaries and mechanisms, not a leaderboard; no number in it is directly comparable across rows. Full comparison in Appendix A.

6 Limitations

Latency is a real, unresolved cost of the no-GPU deployment this paper evaluates. 142 of 159 timed calls (89%) take over 680 ms (p95 1.44 s) on the synchronous hot path (§4.8) — the exact configuration validated throughout §4, not a faster variant. Whether bounding or deferring the background COMPASS/NAVIGATOR audit work closes this gap is asserted, not measured; a real deferred-execution deployment is future work.

No head-to-head with ClawGuard specifically. An earlier version lacked any AgentDojo evaluation at scale; that gap is now closed by §4.6's 629-real-trajectory validation. A narrower gap remains: no controlled head-to-head against ClawGuard's implementation (§5), which intercepts live sessions with no dataset-replay harness compatible with our pipeline.

External tool-call-content baselines are structurally inapplicable, not weaker. We attempted comparison against the two closest available external tools. Both proved structurally inapplicable rather than merely weaker: CodeShield's supported-language set (C/C++/C#/Java/JavaScript/PHP/Python/Rust) excludes the shell, PowerShell, and cmd dialects comprising ∼87% of our reference corpus's adversary-emulation commands, and gitleaks — a secret-value scanner — produces zero findings on all 519 cases, since they encode credential-access techniques rather than embedded literal secrets. We report this as corroborating the enforcement-boundary gap this paper targets, not a performance comparison.

Tamper-evident, not tamper-proof. The hash-chained ledger detects post-hoc edits to any recorded decision or its provenance, but an attacker with write access to the database can recompute the entire chain; defeating that requires anchoring the chain head to an external append-only medium, which we do not implement. We found and fixed a real bug in the ledger's own *verification* query during this work (§3.7).

Curated corpus and deterministic rule set. Both the cosine-triage corpus and the deterministic prefilters' rule sets are authored against published taxonomies (MITRE ATT&CK/ATLAS, AtomicRedTeam, GTFOBins) and known attack corpora (InjecAgent, AgentDojo); a genuinely novel technique outside both may not be caught (§6.1).

The reserve_hotel gap is representative of a broader, unfixed class. §4.6 categorizes the 19 genuine real-trajectory misses in full: five (`reserve_hotel`) are a scoped CHANNEL destination-tool gap whose natural fix we tried and reverted (new false-positive class); the remaining fourteen have no destination value to trace at all, structurally not closeable by any refinement of CHANNEL's mechanism — the shape of question §6 reports as unresolved.

Single embedding model. The cosine-triage layer and NAVIGATOR use a single encoder (`bge-base-en-v1.5`), selected

for open-source offline reproducibility rather than by a comparative ablation; whether an alternative open-source retrieval encoder would shift the residual detection rate is an open question we leave to future work.

PowerShell's real AST is correct but too slow for the live path. §3.3 reports why PowerShell calls are scored by the structural tokenizer rather than a real AST live; a faster in-process PowerShell AST is a scoped next step we did not pursue this cycle.

NAVIGATOR authorization layer: a hard-constraint layer still in progress, after three failed heuristics. NAVIGATOR is the one minister whose deterministic rearchitecture is not yet complete. A direct deterministic swap mirroring VAULT/EXECUTOR/CHANNEL showed zero net discrimination gain and was abandoned; a second attempt reframes the question as authorization rather than similarity (a pinned, immutable session intent, classifying each tool as observational or consequential, then a hard-constraint layer deciding whether a consequential action is *derivable* from that intent). The first two pieces are complete and validated, including under the audit below. The hard-constraint layer failed three independently designed heuristics against a frozen discriminability criterion: (1) tool-name/instruction-text overlap, evaded by attacker text sharing surface words with the legitimate request; (2) argument-value overlap alone, wrongly flagging legitimate values read earlier in the session; (3) a provenance-aware version of (2), still over the frozen threshold.

A proposed alternative for NAVIGATOR: intent-drift, explored and not validated. Independently of the hard-constraint layer above, we explored a second candidate for NAVIGATOR — flagging a call as drift when its embedding similarity to the session's original user-stated goal falls below a threshold — which reached 95.2% recall at 0% FPR on an initial 100-case dataset (83 attack, 17 benign), but did not generalize: enlarging the benign sample to 39 real multi-step AgentDojo sessions raised FPR to 35.9% at the same threshold, since ordinary mid-session follow-ups (e.g., checking a balance right after a payment) are semantically distant from the session's *original* goal despite being legitimate, and held-out recall on the enlarged set collapsed to 22–56%. This is consistent with an independent evaluation of the same mechanism on AgentDojo ground-truth, which measured 7.3% recall. We report this as future work, not a contribution: NAVIGATOR's deployed detector remains the cosine-corpus mechanism in §6.1, unchanged.

No adaptive-attacker evaluation, and a first-pass obfuscation check. Every attack corpus here is static: no attacker knows Kavach's rules or adapts to them. Deterministic matching is historically evadable — encoding, string construction, variable indirection — and while bashlex's AST closes a class of these for sh/bash, PowerShell and cmd remain on a tokenizer. First-pass check: 335 previously-detected sh/bash cases through three transforms (base64 encode-and-pipe, variable-split, variable indirection) held at 100% (0/335 missed) overall. After correctly distinguishing genuine evasion from TRAJECTORY's session-level re-confirmation merely relabeling an already-correct verdict (§3.8), true per-mechanism evasion is 7.0% (encode, 13/185), 4.0% (split, 3/75), 25.3% (indirect, 19/75); every evasion was caught by a different mechanism. One backup is itself imprecise: encode's backstop is a single NAVIGATOR pattern matching the literal `echo BASE64 | base64 -d | sh` shape, confirmed to also block an equivalent benign command — a blanket rule, not intent-aware detection. Extended to PowerShell/cmd (263 detected cases, dialect-native transforms — `-EncodedCommand`, caret-escaping, split/indirection): true evasion is markedly higher, 28.9% (encode, 76/263), 16.4% (split, 12/73), 19.2% (indirect, 14/73) — confirming the tokenizer-not-AST hypothesis empirically. Still a first pass, not adversarial; an attacker targeting two or more mechanisms at once is untested, and §4's rates should be read as non-adaptive until then.

6.1 Generalization beyond the benchmark vocabulary

Kavach's deterministic rules were, in large part, authored by mining detection misses on the exact benchmarks evaluated against (§4.2), a real risk of circular validation. We therefore ran an independent adversarial audit before submission against a fictional agent environment ("AtriumOS") with an entirely invented tool vocabulary sharing no tool name, corpus term, or surface phrasing with InjecAgent, AgentDojo, or our reference corpus: 18 attack and 18 matched-benign novel-vocabulary scenarios through the real pipeline. Summary verdict: EXECUTOR generalizes cleanly, VAULT mostly does (one architecture-inherent limit), CHANNEL's escalation path generalizes but its hard-block path does not, and NAVIGATOR's cosine mechanism does not transfer at all. A second, genuinely independent check against AgentHarm's (UK AISI, ICLR'25) published synthetic tools corroborates this: 81.8% in-scope catch rate (36/44), 8 named misses, and a benign Escalate rate matching §3.8's existing disclosure. Full methodology and per-minister results in Appendix B.

7 Conclusion

Kavach demonstrates that deterministic parsing over a tool call's resolved arguments outperforms cosine similarity over its semantic embedding when the attack signal is structural, not a property of the paradigm as a whole: three of four ministers moved to deterministic-primary detection, while the fourth's intent-shaped question remains open past a validated taxonomy classifier. Validating against 629 real recorded attack trajectories surfaced both real detection strength and precisely characterized gaps. We take this paper's negative results — the falsified fifth minister, the abandoned swap, the failed authorization heuristics, and the generalization audit's boundaries — to be as much a contribution as the numbers: knowing a monitor's limits is what makes its results trustworthy.

A Extended Related Work Comparison

Table 1: Comparison of agent guardrails on five axes. "Boundary" indicates where in the agent lifecycle enforcement fires. "Mechanism" is the primary detection approach. "Ext." indicates whether the detection corpus / rule set is extensible without retraining. "Verdict" indicates how multiple detection signals are combined. AgentDojo ASR and InjecAgent Recall are the best numbers reported by the respective papers; *n/r* = not reported; *ext.* = experimental; † = estimated from paper description; ‡ = cited from the source paper as an upper reference point, not independently reproduced here. Lower ASR is better; higher Recall is better.

System	Boundary	Mechanism	Ext.?	Verdict	AgentDojo ASR ↓	InjecAgent Recall ↑
PromptGuard 2	input	DeBERTa classifier	no	threshold	7.5%	n/r
PlanGuard [11]	plan	isolated planner + verifier	no	hard-constraint	n/r	100%‡
IntentGuard [13]	plan	intent trace + origin	no	alert-first	n/r	n/r
Progent [18]	tool-call	progressive allowlist	yes (policy)	policy-match	1.0%‡	n/r
CodeShield	output	Semgrep+regex	yes (rules)	threshold	n/r	n/r
ShieldAgent	tool-call	rule circuits	yes (docs)	probabilistic	n/r	90.1%
PSG-Agent	tool-call	4-component	partial	cross-turn accum.	n/r	n/r
Kavach (ours)	**tool-call**	**determ.+embed.**	**yes (rules+corpus)**	**veto**	**n/a**§	**100% (544/544 DS)**¶
CaMeL	IFC	dual-LLM + taint	no	provable	23% (sec)	n/r
RTBAS	IFC	saliency screener	no	threshold	2% loss only	n/r
DeepContext	session	RNN over embeds	no	binary	n/r	n/r
Trajectory Guard	trajectory	Siamese RAE	no	score	n/r	n/r
garak [10]	input (test)	vuln. probes	yes (probes)	per-probe	n/r	n/r
PyRIT [16]	input (test)	red-team orch.	yes (attacks)	*n/a*	n/r	n/r

Veto verdict (Kavach): any single minister returning BLOCK vetoes the call unconditionally; no score averaging or learned aggregation. **Corpus extensibility** (Kavach): deterministic prefilters are extended by adding a structural rule; the cosine-triage corpus by adding a natural-language attack-pattern description, re-embedded without retraining. ¶DS = data-stealing subset; direct-harm subset is an open gap (§4.7). §Kavach deliberately reports per-mechanism trajectory analysis (§4.6) rather than a single AgentDojo ASR. 13 additional systems spanning input-time, plan-time, information-flow, and drift-detection guardrails were surveyed (AlignmentCheck, LlamaFirewall combined, AgentSpec, Task Shield, OpenClaw PRISM, FIDES, AGrail, TaskTracker, DRIFT, LLM-Guard, DKnownAI Guard, Lakera Guard, AWS Bedrock); omitted from the table for space.

B Generalization Audit Methodology and Full Results

Why we audit this, and how. Kavach's deterministic detection rules were, in large part, authored by mining detection misses on the exact benchmarks we evaluate against (§4.2), creating a real risk of circular validation: testing detection against the same data shape it was derived from will always flatter the detector. We therefore ran an independent, adversarial generalization audit before submission: a fictional agent environment ("AtriumOS," a smart-building/industrial-facility control agent) with an entirely invented tool vocabulary (`FacilityConfigRead`, `HvacDiagnostic`, `RetrieveTenantAccessLog`, and so on) sharing no tool name, corpus term, or surface phrasing with InjecAgent, AgentDojo, or our reference corpus. We dispatched 18 attack and 18 matched-benign novel-vocabulary scenarios, one per minister's intended detection shape, through the real detection pipeline. This is in tension with our own argument against self-authored attack data (§4.2), which we consider acceptable here because the audit tests *transfer* of mechanisms already built and locked before AtriumOS existed, not a headline detection claim. We report the findings without softening. The 18 attack scenarios are not evenly split across ministers (8 EXECUTOR, 7 VAULT, 3 combined for CHANNEL and NAVIGATOR); EXECUTOR's and VAULT's verdicts below rest on a reasonable sample, but CHANNEL's and NAVIGATOR's rest on very few cases each, and should be read as illustrative rather than statistically powered.

- **EXECUTOR — generalizes cleanly.** Caught all eight novel-vocabulary execution attacks via structural/technique-level shape matching, independent of tool identity. One structural false positive was found (a network-fetch-to-disk predicate firing on a benign data-retrieval call) and fixed pre-submission, with zero new benign false positives on either population; parser test suites remain green (27/27, 37/37).
- **VAULT — mostly generalizes, one inherent limit.** Caught 5/7 novel-vocabulary credential attacks via structural credential-path and secret-format matching. One miss is a narrow surface-form gap (fixable); the other is architectural, not a bug: a named-service secret-key rule is, by construction, a literal allowlist, so a genuinely novel secret-variable name cannot match it regardless of technique.
- **CHANNEL — split, since partially closed.** Its provenance/escalation path generalizes: a behavior-general destination-value fallback correctly escalated a novel send-tool's attacker email address, classified NOVEL. Its taint/hard-block path originally did not, since destination status was decided by a benchmark-specific tool-name dictionary alone; we have since wired the same shape-based fallback into the hard-block path (verified: a novel tool with a shape-matched value, e.g. an IBAN, now correctly hard-BLOCKs), closing the gap for destination values matching a known shape. A genuinely novel exfiltration tool whose destination is *not* shape-matchable (e.g. a

proper-noun business name, the `reserve_hotel` case elsewhere in this paper) remains invisible to the hard-block path — the fix is real but narrower than full generalization.

- **NAVIGATOR — does not currently generalize as a detector.** Cosine similarity against the attack-pattern corpus separates novel attack from benign by only 0.045 (0.590 vs. 0.545 mean), and the BM25 lexical gate that preserves on-benchmark precision makes this worse off-benchmark, collapsing both to ∼0.38 — below the escalation floor, so NAVIGATOR goes silent on novel-domain attacks. This is structural, not incidental: the same mechanism giving NAVIGATOR its low on-benchmark FPR removes its off-benchmark signal. The one general component built for NAVIGATOR — an observational/consequential tool classifier — correctly labeled all four novel tools from verb-token structure alone, but is a *taxonomy, not a detector*: it does not decide whether a consequential action was *authorized*, which requires the pinned-intent authorized-flow layer reported as an incomplete negative result in §6.

A second, genuinely independent check: AgentHarm. AtriumOS is author-written; to test transfer against data this team did not author at all, we used AgentHarm's (UK AI Safety Institute, ICLR 2025) published synthetic tool implementations — 80 harmful-scenario and 80 benign-scenario Python tool files, MIT-licensed for safety research, fetched directly rather than through AgentHarm's official gated benchmark (Inspect framework, API keys, and not tool-call structured regardless). A local `qwen2.5:7b` generated one realistic call per tool from its docstring (a first heuristic attempt was discarded: bland placeholders on the harmful side, suspicious-looking placeholders on the benign side, neither a fair test). Every one of the 79 harmful cases was hand-classified, once, before computing any number, into in-scope for Kavach's stated four-domain threat model or not: AgentHarm covers 11 harm categories far broader than Kavach's claim, and counting e.g. a drug-marketplace search against Kavach would be unfair to both sides. Of 44 in-scope cases: 36 caught (81.8%), 8 missed — anti-forensics (`delete_access_logs`), public exploit-bragging, phishing-asset generation, a phishing email with a malware attachment, a likely tool-name vocabulary gap (`transfer_money_crypto`, the same class of gap as CHANNEL's hard-block path above), two malware-named file uploads, and a no-argument credential harvest consistent with an earlier, independent finding. The 80 benign cases: 0% hard-block, 56.3% Escalate, consistent with §3.8's disclosed 67.3% agent-shaped rate rather than contradicting it.

Concrete next steps. Two boundaries have scoped remedies: a behavior-general destination detector for CHANNEL's hard-block path (mirroring the fallback its provenance path already uses), and a genuinely deterministic authorization layer for NAVIGATOR that converts its already-correct taxonomy classifier into real, vocabulary-independent detection. We frame this section as a strength, not an apology: it is the output of a deliberate, independent validation pass run *before* submission, specifically to surface these boundaries ourselves.

C Provenance Ledger

Every verdict is logged to an append-only SQLite ledger with a SHA-256 hash chain: each row's `entry_hash` is computed over the previous row's hash plus a canonical serialization of that row's content-bearing fields, so editing, deleting, or reordering any historical row breaks every subsequent hash, detectable in $O(n)$ by `/ledger/verify`. The chain is tamper-*evident*, not tamper-*proof*: an attacker with database write access can recompute the whole chain, and defeating that requires an external append-only anchor, left as future work. The hashing scheme is schema-evolution-tolerant — a field added after the ledger was already in production only enters the hash for rows written after it existed, so old rows continue to verify under the same code as new ones. We rely on this directly: during this work we found and fixed a case where the *verification* query, not the ledger itself, omitted a newer column from its recomputation, producing a false tamper report; correcting the query restored a clean `intact: true` across all rows — evidence the mechanism is exercised in practice, not merely specified.

References

[1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. 2024. Are you still on track!? Catching LLM Task Drift with Activations. arXiv:2406.00799 [cs.CR]

[2] Laksh Advani. 2026. Trajectory Guard – A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI. AAAI 2026 Trustworthy Agentic AI Workshop.. arXiv:2601.00516 [cs.LG]

[3] Justin Albrethsen, Yash Datta, Kunal Kumar, and Sharath Rajasekar. 2026. DeepContext: Stateful Real-Time Detection of Multi-Turn Adversarial Intent Drift in LLMs. arXiv:2602.16935 [cs.AI]

[4] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. arXiv:2503.22738 [cs.CR]

[5] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. *arXiv preprint arXiv:2505.03574* (2025).

[6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2026. Securing AI Agents with Information-Flow Control. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. Introduces FIDES.. arXiv:2505.23643

[7] CrowdSec. 2026. CVE-2025-34291 Exploited in the Wild: LangFlow AI Framework Under Fire. https://www.crowdsec.net/vulntracking-report/cve-2025-34291. Active exploitation observed beginning 23 Jan 2026..

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. arXiv:2503.18813 [cs.CR]

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information*

Processing Systems (NeurIPS) Datasets and Benchmarks Track. arXiv:2406.13352 [cs.CR]
[10] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A Framework for Security Probing Large Language Models. NVIDIA. Software: https://github.com/NVIDIA/garak. garak = Generative AI Redteaming and Assessment Kit.. arXiv:2406.11036 [cs.CL]
[11] Guangyu Gong and Zizhuang Deng. 2026. PlanGuard: Defending Agents against Indirect Prompt Injection via Planning-based Consistency Verification. *arXiv preprint arXiv:2604.10134* (2026). arXiv:2604.10134
[12] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. arXiv:2412.16682 [cs.CR]
[13] Mintong Kang, Chong Xiang, Sanjay Kariyappa, Chaowei Xiao, Bo Li, and Edward Suh. 2025. Mitigating Indirect Prompt Injection via Instruction-Following Intent Analysis. *arXiv preprint arXiv:2512.00966* (2025). arXiv:2512.00966
[14] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. arXiv:2407.01502 [cs.LG]
[15] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. 2025. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*. arXiv:2506.12104
[16] Microsoft AI Red Team. 2024. PyRIT: Python Risk Identification Tool for generative AI. https://github.com/Azure/PyRIT. Microsoft red-teaming framework..
[17] Protect AI. 2024. LLM Guard: a security toolkit for LLM interactions. https://github.com/protectai/llm-guard. Open-source input/output scanner suite..
[18] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025). arXiv:2504.11703
[19] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. arXiv:2501.17070 [cs.CR]
[20] Yaozu Wu, Jizhou Guo, Dongyuan Li, Henry Peng Zou, Wei-Chieh Huang, Yankai Chen, Zhen Wang, Weizhi Zhang, Yangning Li, Meng Zhang, Renhe Jiang, and Philip S. Yu. 2025. PSG-Agent: Personality-Aware Safety Guardrail for LLM-based Agents. arXiv:2509.23614 [cs.CR]
[21] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. arXiv:2403.02691 [cs.CL]
[22] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv preprint arXiv:2604.11790* (2026).
[23] Peter Yong Zhong, Siyuan Chen, and Others. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. arXiv:2502.08966 [cs.CR]

Use of Generative AI

Generative AI tools (Anthropic's Claude, via Claude Code) were used throughout this project's development: for implementing detection mechanisms based on the authors' architectural decisions, running and analyzing benchmark evaluations, drafting sections of this paper's text based on the authors' findings and direction, and assisting with code review and debugging. All experimental design choices, architectural decisions, evaluation methodology, and interpretation of results were directed and verified by the authors.